```
import pandas as pd
import re
```

```
from google.colab import drive
##Sorce Files
drive.mount('/content/drive')
print(f"the foolowing csv files are mounted at /content/drive")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_rem
the foolowing csv files are mounted at /content/drive
```

```
df = pd.read_csv('/content/drive/MyDrive/wods_with_category.csv')
df.head()
```

	id	wod	category
0	0	2024 crossfit open workout #1\n\n\nopen 24.1\nfor time:\n21 dumbbell snatches (arm 1) (50/35 lb)\n21 lateral burpees over dumbbell\n21 dumbbell snatches (arm 2) (50/35 lb)\n21 lateral burpees over dumbbell\n15 dumbbell snatches (arm 1) (50/35 lb)\n15 lateral burpees over dumbbell\n15 dumbbell snatches(arm 2) (50/35 lb)\n15 lateral burpees over dumbbell\n9 dumbbell snatches (arm 1) (50/35 lb)\n9 lateral burpees over dumbbell\n9 dumbbell snatches (arm 2) (50/35 lb)\n9 lateral burpees over dumbbell\n\ntime cap: 15 minutes	for_time
1	1	2024 crossfit open workout #2\n\n\nopen 24.2\namrap in 20 minutes of:\n300 meter row\n10 deadlifts (185/125 lb)\n50 double-unders	amrap
2	2	2024 crossfit open workout #3\n\n\nopen 24.3\nfor time\n\n5 rounds of:\n10 thrusters (95/65 lb)\n10 chest-to-bar pull-ups\n\n1 minute rest, then:\n\n5 rounds of:\n7 thrusters (135/95 lb)\n7 bar muscle-ups\n\ntime cap: 15 minutes	for_time
3	3	2021 crossfit games open workout #3 & #4\n\n\nopen 21.3 & 21.4\n"open 21.3"\nfor time\n15 front squats (95/65 lb)\n30 toes-to-bars\n15 thrusters (95/65 lb)\n\nrest 1 minute\n\n15 front squats (95/65 lb)\n30 chest-to-bar pull-ups\n15 thrusters (95/65 lb)\n\nrest 1 minute\n\n15 front squats (95/65 lb)\n30 bar muscle-ups\n15 thrusters (95/65 lb)\n\ntime cap: 15 minutes\n\n"open 21.4"\nmove immediately to complete the following complex for max load:\n\n1 deadlift\n1 clean\n1 hang clean\n1 jerk\n\ntime	other

Next steps: Generate code with df New interactive sheet

```
import re
import pandas as pd


#Data Cleaning
def clean_wod(workout_description):
    description = str(workout_description).lower()
    description = description.split("\n\n")[0]
    description = description.split("\n", 1)[-1]
    format_type = "OTHER"
    if "amrap" in description:
        format_type = "AMRAP"
    elif "for time" in description:
        format_type = "FOR_TIME"
    elif "emom" in description:
        format_type = "EMOM"

    description = re.sub(r"crossfit.*?", "", description)
    description = re.sub(r"open \d+\.\d+", "", description)
    description = re.sub(r"hero wod.*?", "", description)
    description = re.sub(r"benchmark wod.*?", "", description)
    description = re.sub(r"fitness.*?", "", description)
    description = re.sub(r"subscribe.*", "", description)
    description = re.sub(r"coach.*", "", description)
    description = re.sub(r"friend.*", "", description)
    description = re.sub(r"ideas..*", "", description)
    description = re.sub(r"[^\w\s<>-]", "", description)
    description = re.sub(r"\b\d{6,8}\b", "", description)
    description = re.sub(r"e\d*mom", "emom", description)
    description = re.sub(r"(\d+)[/](\d+)", r"\\1_\\2", description)
    description = re.sub(r"advanced filters find the wod you want go beastmode", "", description)
    description = re.sub(r"featured", "", description)
    description = re.sub(r"aka macc", "", description)
    description = re.sub(r"nightmare 4", "", description)

    description = re.sub(
        r"move immediately.*?max load:",
        "",
```

```
        description
    )

    description = re.sub(r"\b20\d{2}\b", "", description)
    description = re.sub(r"\b\d+\.\d+\b", "", description)
    description = re.sub(r"#\d+", "", description)
    description = re.sub(r"\bgames\b|\bopen\b|\bworkout\b", "", description)
    description = re.sub(r"\(.*?\)", "", description)
    description = re.sub(r"rounds of:", "rounds", description)
    description = re.sub(r"then:", "", description)
    description = re.sub(r"(\d+)\s*meter", r"\\1m", description)
    match = re.search(r"(\d+)\s*(minutes|min)", description)
    duration = match.group(1) if match else '' # removing nulls
    description = re.sub(r"\d+\s*(minutes|min)", "", description)
    description = re.sub(r"\bamrap\b|\bfor time\b|\bemom\b", "", description)
    description = re.sub(r"&+", " ", description)
    description = re.sub(r'"+', " ", description)

    # standardizing movements
    replacements = {
        "deadlifts": "deadlift",
        "double-unders": "double-under",
        "single-unders": "single-under",
        "pull-ups": "pull-up",
        "pullup": "pull-up",
        "push-ups": "pushup",
        "sit-ups": "situp ",
        "back-squats": "back squat",
        "front-squats": "front squat",
        "front squats": "front squat",
        "overhead-squats": "overhead squat",
        "overhead-squat": "overhead squat",
        "side-lunges": "side lunge",
        "lunges": "lunge",
        "squats": "squat",
        "bench-press": "bench press",
        "overhead-press": "overhead press",
        "toes-to-bars": "toe-to-bar",
        "toes to bar": "toe-to-bar",
        "thrusters": "thruster",
        "muscle-ups": "muscle-up",
        "bar muscle ups": "bar muscle-up",
        "hang clean": "hang-clean",
    }

    for old, new in replacements.items():
        description = description.replace(old, new)


    description = re.sub(r"\n+", " ", description)
    description = re.sub(r"\s+", " ", description).strip()
    description = re.sub(r"^[a-z\s]+(?=\d)", "", description)

    return format_type, duration, description

# Apply the clean_wod function to create new columns
df[['format', 'duration', 'clean_text']] = df['wod'].apply(lambda x: pd.Series(clean_wod(x)))

# function for  output
def build_model_text(row):
    return (
        f"<FORMAT> {row['format']} "
        + (f"<DURATION> {row['duration']} " if row["duration"] else "")
        + f"<WORKOUT> {row['clean_text']}"
    )

df["model_text"] = df.apply(build_model_text, axis=1)

# More cleaning
df = df[df["clean_text"].str.split().str.len() > 8]
df = df[df["clean_text"].str.split().str.len() >= 6]
df = df[~df["clean_text"].str.contains(r"sa1dbagg|ute rest|WORKOU|^\d+$", na=False)]


df.to_csv("wods_cleaned_for_model.csv", index=False)

pd.set_option("display.max_colwidth", None)
#print(df["model text"].head(10))
```

```
#saving File
df.to_csv("wod2_cleaned_production.csv", index=False)
```

```
model_output = df["model_text"]
model_output.to_csv("model_output.csv", index=False)
```

Preparing the Model

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from collections import Counter

#removing the other column
df = df[df["format"].isin(["AMRAP", "EMOM", "FOR_TIME"])]


# reading in data
texts = df["model_text"].dropna().astype(str).tolist()


# Setting Hyperparameters
hidden_size = 128
learning_rate = 0.01
num_epochs = 200
teacher_forcing_ratio = 0.5

#tokenizer
def tokenize(text):
    return text.strip().split()

tokenized_texts = [tokenize(text) for text in texts]


# Building vocabulary
special_tokens = ["<PAD>", "<SOS>", "<EOS>", "<UNK>"]

counter = Counter()
for tokens in tokenized_texts:
    counter.update(tokens)

vocab = {token: idx for idx, token in enumerate(special_tokens)}
for token in counter:
    if token not in vocab:
        vocab[token] = len(vocab)

idx_to_token = {idx: token for token, idx in vocab.items()}

PAD_IDX = vocab["<PAD>"]
SOS_IDX = vocab["<SOS>"]
EOS_IDX = vocab["<EOS>"]
UNK_IDX = vocab["<UNK>"]

vocab_size = len(vocab)

print("Vocabulary size:", vocab_size)
print("Vocabulary:", vocab)

# Convert text to tensor
def text_to_tensor(text, vocab):
    tokens = tokenize(text)
    token_ids = [vocab.get(token, UNK_IDX) for token in tokens]
    token_ids = [SOS_IDX] + token_ids + [EOS_IDX]
    return torch.tensor(token_ids, dtype=torch.long)

sequences = [text_to_tensor(text, vocab) for text in texts]

print("\nExample tokenized sequence:")
print(sequences[0])

# Decoder
```

```
class DecoderRNN(nn.Module):
    def __init__(self, hidden_size, output_size):
        super().__init__()
        self.hidden_size = hidden_size
        self.embedding = nn.Embedding(output_size, hidden_size)
        self.gru = nn.GRU(hidden_size, hidden_size)
        self.out = nn.Linear(hidden_size, output_size)
        self.log_softmax = nn.LogSoftmax(dim=1)

    def forward(self, input_token, hidden):
        embedded = self.embedding(input_token).view(1, 1, self.hidden_size)
        output, hidden = self.gru(embedded, hidden)
        output = self.log_softmax(self.out(output[0]))
        return output, hidden

decoder = DecoderRNN(hidden_size, vocab_size)
optimizer = optim.SGD(decoder.parameters(), lr=learning_rate)
criterion = nn.NLLLoss()

# Training loop
for epoch in range(num_epochs):
    total_loss = 0

    for seq in sequences:
        optimizer.zero_grad()

        decoder_hidden = torch.zeros(1, 1, hidden_size)

        input_seq = seq[:-1]

        target_seq = seq[1:]

        loss = 0
        decoder_input = input_seq[0].unsqueeze(0)

        for t in range(len(target_seq)):
            decoder_output, decoder_hidden = decoder(decoder_input, decoder_hidden)
            loss += criterion(decoder_output, target_seq[t].unsqueeze(0))

            use_teacher_forcing = torch.rand(1).item() < teacher_forcing_ratio

            if use_teacher_forcing and t + 1 < len(input_seq):
                decoder_input = input_seq[t + 1].unsqueeze(0)
            else:
                topv, topi = decoder_output.topk(1)
                decoder_input = topi.squeeze().detach().unsqueeze(0)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    if (epoch + 1) % 20 == 0:
        avg_loss = total_loss / len(sequences)
        print(f"Epoch {epoch+1}/{num_epochs}, Avg Loss: {avg_loss:.4f}")

# Text Generator
def generate_text(decoder, vocab, idx_to_token, max_length=20, start_token="<SOS>", temperature=0.8):
    decoder.eval()

    with torch.no_grad():
        decoder_hidden = torch.zeros(1, 1, hidden_size)
        decoder_input = torch.tensor([vocab[start_token]], dtype=torch.long)

        generated_tokens = []

        for _ in range(max_length):
            decoder_output, decoder_hidden = decoder(decoder_input, decoder_hidden)

            # Convert log probs to probabilities and apply temperature
            probs = torch.exp(decoder_output / temperature).squeeze()
            probs = probs / probs.sum()

            # Sample next token from the distribution
            next_token_idx = torch.multinomial(probs, 1).item()

            if next_token_idx == EOS_IDX:
```

```
                break

            generated_tokens.append(idx_to_token[next_token_idx])
            decoder_input = torch.tensor([next_token_idx], dtype=torch.long)

    return " ".join(generated_tokens)

# 10. Test generation
print("\nGenerated workout:")
print(generate_text(decoder, vocab, idx_to_token))
```

```
Vocabulary size: 523
Vocabulary: {'<PAD>': 0, '<SOS>': 1, '<EOS>': 2, '<UNK>': 3, '<FORMAT>': 4, 'FOR_TIME': 5, '<WORKOUT>': 6, 'wyck': 7,

Example tokenized sequence:
tensor([ 1,  4,  5,  6,  7,  8,  9, 10, 11,  8,  9, 12, 13, 14, 15, 16,  8,  9,
        17, 18, 19, 20, 16,  8,  9, 17, 18, 19, 20, 16,  8,  9, 12, 13, 14, 15,
        16,  8,  9, 10, 11,  8,  9,  2])
Epoch 20/200, Avg Loss: 65.4469
Epoch 40/200, Avg Loss: 31.1584
Epoch 60/200, Avg Loss: 27.3344
Epoch 80/200, Avg Loss: 24.4309
Epoch 100/200, Avg Loss: 23.0437
Epoch 120/200, Avg Loss: 20.6389
Epoch 140/200, Avg Loss: 22.5669
Epoch 160/200, Avg Loss: 22.1809
Epoch 180/200, Avg Loss: 20.0305
Epoch 200/200, Avg Loss: 20.5718

Generated workout:
<FORMAT> FOR_TIME <WORKOUT> dumbbell run \1m run 20 dumbbell clean-and-presses 2x5035 lb \1m run 20 dumbbell thruster
```

```
import re

#Cleaning Final output
def clean_generated_workout(text):
    text = text.lower()

    text = re.sub(r"<format>", "<FORMAT>", text)
    text = re.sub(r"<duration>", "<DURATION>", text)
    text = re.sub(r"<workout>", "<WORKOUT>", text)
    text = re.sub(r"(<WORKOUT>\s*)+", "<WORKOUT> ", text)
    text = re.sub(r"\bute\b", "minute", text)
    text = re.sub(r"\b(\d{2})(\d{2})\s*lb\b", r"\1/\2 lb", text)
    text = re.sub(r"\b(\d{2})(\d{2})lb\b", r"\1/\2 lb", text)

    text = re.sub(r"\b(\d{3})(\d{2})\s*lb\b", r"\1/\2 lb", text)
    text = re.sub(r"\b(\d{3})(\d{2})lb\b", r"\1/\2 lb", text)

    text = re.sub(r"\b(\d{3})(\d{3})\s*lb\b", r"\1/\2 lb", text)
    text = re.sub(r"\b(\d{3})(\d{3})lb\b", r"\1/\2 lb", text)

    text = re.sub(r"\b(\d{2})(\d{2})\s*kg\b", r"\1/\2 kg", text)
    text = re.sub(r"\b(\d{2})(\d{2})kg\b", r"\1/\2 kg", text)
    text = re.sub(r"\bwall ball shots lb (\d{2})(\d)\b", r"wall ball shots lb \1/\2", text)
    text = re.sub(r"\bbench presses (\d{3})(\d{2})\b", r"bench presses \1/\2 lb", text)
    text = re.sub(r"\bbear complex (\d{3})(\d{2})\b", r"bear complex \1/\2 lb", text)
    text = re.sub(r"\bhang-clean (\d{2})(\d{2})\b", r"hang-clean \1/\2 lb", text)
    text = re.sub(r"\bsandbag cleans (\d{2})(\d{2})\b", r"sandbag cleans \1/\2 lb", text)
    text = re.sub(r"\bsled push (\d{3})(\d{3})\b", r"sled push \1/\2 lb", text)
    text = re.sub(r"\bbox overs (\d{2})(\d{2})\b", r"box overs \1/\2", text)
    text = re.sub(r"\bdb curl press (\d{2})(\d{2})\b", r"db curl press \1/\2 lb", text)
    text = re.sub(r"\bgoblet squat (\d{2})(\d{2})\b", r"goblet squat \1/\2 lb", text)

    text = re.sub(r"\bdumbbell deadlift (\d{2})(\d{2})\s*lb\b", r"dumbbell deadlift \1/\2 lb", text)
    text = re.sub(r"\bdumbbell squat cleans (\d{2})(\d{2})\s*lb\b", r"dumbbell squat cleans \1/\2 lb", text)
    text = re.sub(r"\bdumbbell shoulder-to-overheads (\d{2})(\d{2})\s*lb\b", r"dumbbell shoulder-to-overheads \1/\2
    text = re.sub(r"\b9565\b", "95/65", text)
    text = re.sub(r"\b11575\b", "115/75", text)
    text = re.sub(r"\b(\d{2})(\d{2})\b", r"\1/\2", text)

    text = re.sub(r"\b(\d{3})(\d{2})\b", r"\1/\2", text)
    text = re.sub(r"\b2x(\d{2})(\d{2})\b", r"2x\1/\2", text)
    text = re.sub(r"\b2x(\d{3})(\d{2})\b", r"2x\1/\2", text)
    text = re.sub(r"(<WORKOUT>\s*)+", "<WORKOUT> ", text)

    text = re.sub(r"\balternating dumbbell deadlift (\d{2})(\d{2})\s*lb\b", r"alternating dumbbell deadlift \1/\2 lb
    text = re.sub(r"\balternating dumbbell squat cleans (\d{2})(\d{2})\s*lb\b", r"alternating dumbbell squat cleans
```

```
    text = re.sub(r"\balternating dumbbell shoulder-to-overheads (\d{2})(\d{2})\s*lb\b", r"alternating dumbbell shou
    text = re.sub(r"\balternating dumbbell squat (\d{2}/\d{2}) lb\b", r"alternating dumbbell squat cleans \1 lb", te
    text = re.sub(r"\bwall ball shots lb (\d{2}/\d)\b", r"wall ball shots \1 lb", text)

    text = re.sub(r"advanced filters find the wod you want go beastmode", "", text)
    text = re.sub(r"with a running clock", "", text)
    text = re.sub(r"essential worker", "", text)
    text = re.sub(r"(<WORKOUT>\s*)+", "<WORKOUT> ", text)
    text = re.sub(r"choe in buy-in", "", text)

    text = re.sub(r"\s+", " ", text).strip()

    return text
```

```
#Final Model Output
cleaned_outputs = []

for i in range(1):
    workout = generate_text(decoder, vocab, idx_to_token, temperature=0.8)
    clean_workout = clean_generated_workout(workout)
    cleaned_outputs.append(clean_workout)

    print(f"Workout {i+1}:")
    print(clean_workout)
    print()
```

```
Workout 1:
<FORMAT> amrap <DURATION> 25 <WORKOUT> 10 burpee broad jumps 20 pushup 30 sandbag lunge 60/40 lb 40 calorie row
```